Use just two tables in your mind:

STUDENTS	ENROLLMENTS
1 Arun	1 AWS
2 Priya	2 Azure
3 Rahul	3 GenAI
4 Sneha	

Now remember only these **4 main JOINs**:

1. INNER JOIN = MATCH ONLY
2. LEFT JOIN = ALL LEFT
3. RIGHT JOIN = ALL RIGHT
4. FULL JOIN = ALL BOTH

INNER JOIN = "Match only"

Think: **Both sides must have data.**

```
SELECT *
FROM students s
INNER JOIN enrollments e
ON s.student_id = e.student_id;
```

Result:

```
Arun   AWS
Priya  Azure
Rahul  GenAI
```

Sneha is not coming because she doesn't have an enrollment.

Remember:

INNER = MATCH ONLY

LEFT JOIN = "Keep my left table"

```
SELECT *
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id;
```

Look at the query:

```
students  LEFT JOIN  enrollments
  ↑
 KEEP
```

Result:

```
Arun    AWS
Priya   Azure
Rahul   GenAI
Sneha   NULL
```

Even though Sneha doesn't have enrollment, she comes because students is on the **LEFT**.

Remember:

LEFT JOIN = ALL LEFT

RIGHT JOIN = "Keep my right table"

```
SELECT *
FROM enrollments e
RIGHT JOIN students s
```

ON e.student_id = s.student_id;
Look:
enrollments RIGHT JOIN students
↑
KEEP
Result:
AWS Arun
Azure Priya
GenAI Rahul
NULL Sneha
Remember:
RIGHT JOIN = ALL RIGHT
One important trick:
students LEFT JOIN enrollments
is basically the same idea as:
enrollments RIGHT JOIN students
because in both cases we are saying:
"Keep all students."

FULL OUTER JOIN = "Keep everything"
SELECT *
FROM students s
FULL OUTER JOIN enrollments e
ON s.student_id = e.student_id;
Remember:
FULL = ALL LEFT + ALL RIGHT
If something matches, combine it.
If it doesn't match, still show it with NULL.

Now memorize this one line:
INNER = Match, LEFT = Left all, RIGHT = Right all, FULL = Both all.
Think of **CROSS JOIN as "make every possible pair."**
Suppose you have only **2 students**:
STUDENTS

Arun
Priya
And you have **3 courses**:
COURSES

AWS
Azure
Python
Now I ask:
"Show every course to every student."
So Arun gets all 3 courses:
Arun → AWS
Arun → Azure

Arun → Python
Priya also gets all 3:
Priya → AWS
Priya → Azure
Priya → Python
Final result:
Arun AWS
Arun Azure
Arun Python
Priya AWS
Priya Azure
Priya Python
That is **CROSS JOIN**.
The easiest formula is:
LEFT rows × RIGHT rows = Result rows
So:
2 students × 3 courses = 6 rows
In your actual data, you have **12 students** and **9 courses**.

Think of **UNION and UNION ALL as "put one result below another result."**
Forget SQL first.
Suppose Table 1 gives:
Chennai
Bangalore
Hyderabad
Table 2 gives:
Chennai
Pune
Bangalore
Now we want to put Table 2 **under** Table 1.
UNION = combine, but remove duplicates

Table 1	Table 2
Chennai	Chennai
Bangalore	Pune
Hyderabad	Bangalore

↓ UNION ↓

Chennai
Bangalore
Hyderabad
Pune
Notice Chennai and Bangalore came in both results, but UNION shows them only **once**.
SQL:

```
SELECT city
FROM students
WHERE student_id <= 5

UNION

SELECT city
FROM students
WHERE student_id > 5;
```

Just remember:
UNION = PUT TOGETHER + REMOVE DUPLICATES

UNION ALL = combine and keep everything
Same example:

Table 1	Table 2
Chennai	Chennai
Bangalore	Pune
Hyderabad	Bangalore

↓ UNION ALL ↓

Chennai
Bangalore
Hyderabad
Chennai
Pune
Bangalore
Here duplicates are **not removed**.
SQL:

```
SELECT city
FROM students
WHERE student_id <= 5

UNION ALL

SELECT city
FROM students
WHERE student_id > 5;
```

Remember:
UNION ALL = PUT TOGETHER + KEEP ALL

Think of **GROUP BY as "put same values into one group and calculate something."**
Forget SQL first.
Suppose your students are:
Arun Chennai
Priya Chennai
Rahul Bangalore
Sneha Hyderabad
Karthik Chennai
Divya Bangalore
Now I ask:
"How many students are there in each city?"
First SQL mentally creates groups:
Chennai
Arun
Priya
Karthik

Bangalore
Rahul
Divya

Hyderabad
Sneha
Then it counts each group:
Chennai 3
Bangalore 2
Hyderabad 1
That is **GROUP BY**.
SQL:

```
SELECT
   city,
   COUNT(*) AS total_students
FROM students
GROUP BY city;
```

Remember:
GROUP BY = SAME VALUES TOGETHER

Think of **HAVING as "filter the GROUP BY result."**
You already know:
GROUP BY = Same values together
Now add:
HAVING = Filter those groups
Suppose students are:
Arun Chennai
Priya Chennai
Karthik Chennai

Rahul Bangalore
Divya Bangalore

Sneha Hyderabad
First, GROUP BY city:

```
SELECT city, COUNT(*)
FROM students
GROUP BY city;
```

Result:
Chennai 3
Bangalore 2
Hyderabad 1
Now I ask:
“Show only cities having more than 1 student.”
We use HAVING:

```
SELECT
   city,
   COUNT(*) AS total_students
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

Result:
Chennai 3
Bangalore 2
Hyderabad is removed because:
Hyderabad = 1

1 > 1 □
So remember:
GROUP BY
↓
Make groups
↓
COUNT / SUM / AVG
↓
HAVING
↓
Filter the groups
The most important thing is **WHERE vs HAVING**.
WHERE = filter ROWS

HAVING = filter GROUPS
Example:

```
SELECT city, COUNT(*)
FROM students
WHERE experience_years > 1
GROUP BY city
HAVING COUNT(*) > 1;
```

Think about the order:
Students
↓
WHERE experience_years > 1
↓
Remove unwanted individual students
↓
GROUP BY city
↓
Create city groups
↓
COUNT(*)
↓
HAVING COUNT(*) > 1
↓
Remove unwanted groups
So the easiest memory trick is:
WHERE = before grouping
HAVING = after grouping